Design and Implementation of a Web-Based Stock Scoring Engine

Leul Ejigu

February 18, 2026

Introduction

Financial investment in the modern era is characterized by an huge amount of data. Investors seeking to analyze the S&P 500 index must navigate a complex landscape of quantitative metrics such as revenue growth and P/E ratios, alongside qualitative factors, such as leadership stability and competitive advantage. Visualizing data for over 500 companies presents a significant computational and design challenge. To address this, this project proposes the development of a web-based Stock Scoring Engine. This application will digest large financial datasets to produce a ranked list of stocks based on a proprietary 13-metrics scoring system(Thought it is essential for a stock to have) .

The value of this Junior Is is in its approach to information hierarchy. Research by Dong et al. shows that visualizing financial "big data" works best when using specific "post-model" strategies. Instead of dumping raw and confusing data on the user, this system handles the heavy calculations in the backend. It processes the entire S&P 500 and only presents the most important trends and summary stats for the top-ranked stocks. This keeps the dashboard clean and actionable, solving the information overload problem that usually makes financial analysis difficult for individual investors.

Project Goals and Methodology

The primary goal of this software is to automate the ranking of stocks by synthesizing "hard" and "soft" data. Ettredge, Richardson, and Scholz investigate the structure of financial websites, distinguishing between hard data (stock prices, financial statements) and soft information (investor relations, corporate culture). This project adopts their design model to organize the user interface. The proposed system will calculate a composite score out of 130 points, derived from 13 distinct metrics. The interface will be structured to allow users to view the high-level ranking immediately while retaining the ability to drill down into the subjective "soft" scores, such as Moat or Leadership, without obscuring the objective "hard" numbers. To ensure the ranking is accurate and representative of the market, the system adheres to the official methodology outlined by S&P Dow Jones Indices. The application will implement filtering logic based on divisor-based weighting and float-adjustment criteria.

This ensures that the universe of stocks analyzed by the software is consistent with industry standards for the S&P 500, preventing the inclusion of ineligible or low-liquidity assets in the final ranking. The core computational engine of the application will be built using Python, specifically leveraging the pandas library. As described by McKinney, the DataFrame provides the optimal data structure for statistical computing and data alignment. The project relies on the DataFrame structure to efficiently store, index, and vectorize calculations across the 500-row dataset for the 13 required metrics.

Technical Architecture and System Design

The core engine of the application will be built using Python and the pandas library. As described by McKinney [4], the DataFrame is the ideal structure for this kind of statistical computing. The project relies on DataFrames to store, index, and run calculations across the 500-stock dataset efficiently.

A major challenge in this project is automating the qualitative scores. While math-based ratios are easy to calculate, metrics like "News/Catalyst" require analyzing unstructured text. This project builds on the work of Schumaker and Chen [6], who created the AzFin text system to classify financial news. However, instead of building a custom text analysis module from scratch, this system will integrate a modern AI API. This allows the application to process live news feeds and sentiment more accurately .

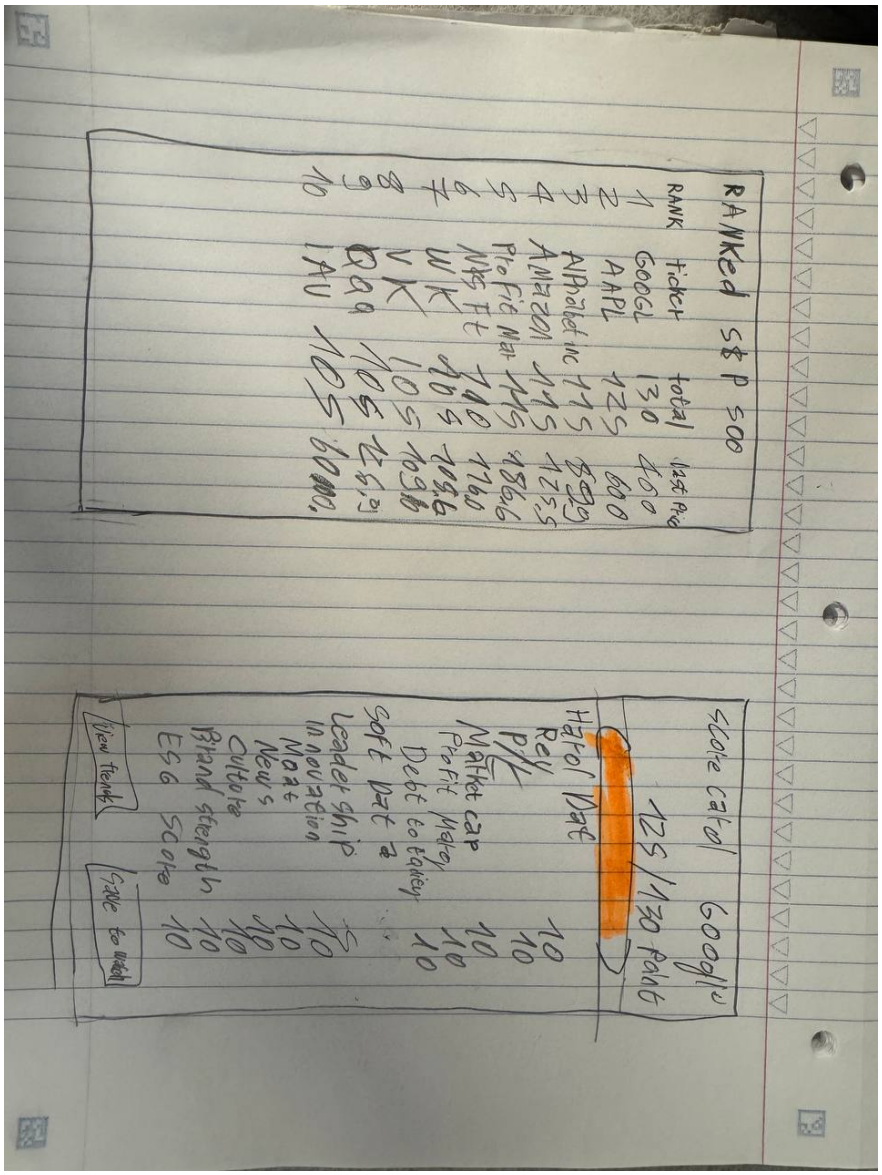

Figure 1: Prototype of the web application interface showing the ranked stock list and detailed metric breakdowns.

Appendix

A concise list of features / user stories in the order in which they will be built. A few examples are below to demonstrate the expected scope and level of granularity; you will have more features than this.

- **Initial Setup:** Set up the Python environment and install pandas and the web framework.
- **Stock List Download:** Create a script to grab all current S&P 500 ticker symbols.
- **Financial Data Ingestion:** Build functions to download raw data (like P/E ratios and revenue) for the 500 stocks.
- **S&P Standards Filter:** Add logic to filter stocks based on official S&P eligibility rules.
- **Data Storage:** Create a pandas DataFrame to hold and organize all stock metrics in one place.
- **Numerical Scoring:** Write the code to turn financial ratios into a score from 1 to 10.
- **AI Qualitative Analysis:** Connect an AI API to read news headlines and score the "soft" metrics.
- **Ranking Engine:** Create a script that adds up all 13 metrics and sorts the stocks from highest to lowest.
- **Data API:** Set up a backend that sends the final ranked data to the web interface.
- **Web Dashboard:** Build the main table where users can see the top-ranked stocks.
- **Detailed View:** Add a feature so users can click a stock to see exactly how it got its score.
- **Stretch Goal:** Add a live search bar to find any stock in the list instantly.
- **Stretch Goal:** Allow users to save their favorite stocks to a personal watchlist.

References

[1] Dong, X., Huang, W., and Wang, J. Financial big data visualization: A machine learning perspective. In *Proceedings of the 17th International Symposium on Visual Information Communication and Interaction (VINCI '24)* (New York, NY, USA, 2024), ACM. Accessed: 2026-01-27.

[2] dos Santos Pinheiro, L., and Dras, M. Stock market prediction with deep learning: A character-based neural language model for event-based trading. In *Proceedings of the Australasian Language Technology Association Workshop 2017* (2017), pp. 6–15. Accessed: 2026-02-19.

[3] Ettredge, M., Richardson, V. J., and Scholz, S. A web site design model for financial information. *Communications of the ACM 44*, 11 (2001), 51–55. Accessed: 2026-01-27.

[4] McKinney, W. Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference* (2010), S. van der Walt and J. Millman, Eds., pp. 56–61. Accessed: 2026-01-27.

[5] Runsewe, O., Akwawa, L. A., Folorunsho, S. O., and Osundare, O. S. Optimizing user interface and user experience in financial applications: A review of techniques and technologies. *World Journal of Advanced Research and Reviews 23*, 03 (2024), 934–942.

[6] Schumaker, R. P., and Chen, H. Textual analysis of stock market prediction using breaking financial news: The azfin text system. *ACM Transactions on Information Systems 27*, 2 (2009), 12:1–12:19. Accessed: 2026-01-27.

[7] S&P Dow Jones Indices. S&p u.s. indices methodology. Tech. rep., McGraw Hill Financial, February 2016. Accessed: 2026-01-27.